#1 Method `longestPrefix` takes a trie node, the string key `s` to be searched, and a string with the path from root to `node`; it returns the longest prefix of `s` stored in the trie.
#2 Check if the string searched is empty: if it is, since this method is implemented using recursion, we know it has traversed the whole path in the trie.
#3 We are at the final node of the path for the searched string: if it's a *key node*, then the string itself (by know accumulated into `prefix`) is its longest prefix in the trie.
#4 Otherwise, we need to return `null` to let the caller know we haven't found any key yet.
#5 Since `s` is not the empty string, it can be broken into a *head* character, `c` (the first character of `s`) and a *tail*, the rest of the string.
#6 If there is no outgoing edge in `node` for the character `c`, it means that we cannot traverse the trie any further; we need to return `null` to let the caller know we haven't found any result in this subtree.
#7 Otherwise, we recursively search `tail` into the subtree referenced by `children[c]`, and store the longest result found in this subtree into a temporary variable.
#8 If we have found a prefix of the searched string stored into the subtree, then it is going to be longer than any other prefix we can find backtracking: we can just propagate the result back.
#9 Otherwise, if we found nothing in the subtree for children[c], we can check if this node is a *key node*, meaning that the path to it is the longest prefix for the searched string.
#10 If the recursive call hasn't found anything and this is not a key node, we return null to let the caller know we don't